

SQL test

#1. What are the 10 most expensive products in the company?

```
SELECT PRODUCT_NAME, PRODUCT_VAL
FROM `loogbox-challenge`.data_product
ORDER BY PRODUCT_VAL DESC
LIMIT 10;
```

PRODUCT_NAME	PRODUCT_VAL
Whisky Escoces THE MACALLAN Ruby Garrafa 700ml com Caixa	741.99
Whisky Escoces JOHNNIE WALKER Blue Label Garrafa 750ml	735.90
Cafeteira Expresso 3 CORACOES Tres Modo Vermelho	499.00
Vinho Portugues Tinto Vintage QUINTA DO CRASTO Garrafa 750ml	445.90
Escova Dental Eletrica ORAL B D34 Professional Care 5000 110v	399.90
Champagne Rose VEUVE CLICQUOT PONSARDIM Garrafa 750ml	366.90
Champagne Frances Brut Imperial MOET Rose Garrafa 750ml	359.90
Conjunto de Panelas Allegra em Inox TRAMONTINA 5 Pecas Grati...	359.00
Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml	329.90
Champagne Frances Brut Imperial MOET & CHANDON Garrafa 750ml	315.90

#2. What sections do the 'BEBIDAS' and 'PADARIA' departments have?

```
SELECT DISTINCT DEP_NAME , SECTION_NAME
FROM `loogbox-challenge`.data_product
WHERE DEP_NAME IN ('BEBIDAS','PADARIA')
ORDER BY DEP_NAME, SECTION_NAME;
```

DEP_NAME	SECTION_NAME
BEBIDAS	BEBIDAS
BEBIDAS	CERVEJAS
BEBIDAS	REFRESCOS
BEBIDAS	VINHOS
PADARIA	DOCES-E-SOBREMESAS
PADARIA	GESTANTE
PADARIA	PADARIA
PADARIA	QUEIJOS-E-FRIOS

#3. What was the total sale of products (in \$) of each Business Area in the first quarter of 2019?

```
SELECT CAD.BUSINESS_NAME , SUM(SALES.SALES_VALUE) AS SALES_VALUE
FROM `looqbox-challenge`.data_store_sales AS SALES
LEFT JOIN `looqbox-challenge`.data_store_cad as CAD
ON SALES.STORE_CODE = CAD.STORE_CODE
WHERE SALES.DATE BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY CAD.BUSINESS_NAME
ORDER BY SUM(SALES.SALES_VALUE) DESC;
```

BUSINESS_NAME	SALES_VALUE
Farma	81776691.73
Varejo	81032347.65
Atacado	80384884.60
Proximidade	80171122.80
Posto	32072326.40

Case 1

1) The Dev Team was tired of developing the same old queries just varying the filters accordingly to their boss demands.

As a new member of the crew, your mission now is to create a dynamic function in Python, on the most flexible of ways, to produce queries and retrieve a dataframe based on three parameters:

- product_code: integer
- store_code: integer
- date: list of ISO-like strings
- Date e.g.
 - ['2019-01-01', '2019-01-31']

It should look like this `my_data = retrieve_data(product_code, store_code, date)`

Make your team proud!

Extra instructions:

- Retrieve all columns from table `data_product_sales`;
- Imagine people from other teams will also utilize this function!

```
In [1]: import pandas as pd
        from sqlalchemy import create_engine
```

```
In [ ]: def retrieve_data(product_code, store_code, date):
        # 1. Credenciais de acesso
        usuario = 'loobox-challenge'
        senha = 'loooq-challenge'
        host = '35.199.115.174'
        porta = '3306'          # porta padrão do MySQL
        nome_do_banco = 'loobox-challenge'

        # 2. String de conexão (URL de conexão)
        # Formato: mysql+pymysql://usuario:senha@host:porta/nome_do_banco
        string_conexao = f'mysql+pymysql://{usuario}:{senha}@{host}:{porta}/{nome_do_banco}'

        # 3. Motor de conexão (engine)
        engine = create_engine(string_conexao)

        # 4. Consulta SQL
        query = """SELECT * FROM `loobox-challenge`.`data_product_sales`
                WHERE PRODUCT_CODE = %s
                AND DATE BETWEEN %s AND %s
                AND STORE_CODE = %s"""

        df = pd.read_sql(query, con=engine, params=(product_code, date[0], date[1], store_code))

        # 6. retorna o DataFrame
        return df
```

```
Out[ ]:
```

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	1	18	2019-01-01	708.5	65.0
1	1	18	2019-01-02	1297.1	119.0
2	1	18	2019-01-03	1144.5	105.0
3	1	18	2019-01-04	1090.0	100.0
4	1	18	2019-01-05	893.8	82.0

```
In [20]: my_data = retrieve_data(product_code=18, store_code=1, date=['2019-01-01', '2019-03-31'])
my_data
```

Out[20]:

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	1	18	2019-01-01	708.5	65.0
1	1	18	2019-01-02	1297.1	119.0
2	1	18	2019-01-03	1144.5	105.0
3	1	18	2019-01-04	1090.0	100.0
4	1	18	2019-01-05	893.8	82.0
...	...	...	...	...	...
85	1	18	2019-03-27	1308.0	120.0
86	1	18	2019-03-28	1079.1	99.0
87	1	18	2019-03-29	1002.8	92.0
88	1	18	2019-03-30	741.2	68.0
89	1	18	2019-03-31	741.2	68.0

90 rows × 5 columns

Case 2

2) A brand new client sent you two ready-to-go queries. Those are listed below:

Query 1:

```
SELECT
    STORE_CODE,
    STORE_NAME,
    START_DATE,
    END_DATE,
    BUSINESS_NAME,
    BUSINESS_CODE
FROM data_store_cad
```

Query 2:

```
SELECT
    STORE_CODE,
    DATE,
    SALES_VALUE,
    SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
```

In addition, he gave you this set of instructions:

- Use the queries as they are (do not modify them or create a new one);
- Please filter the period between this given range:
 - ['2019-10-01','2019-12-31']

► We are in need of this visualization (click here to see it)! Please, create it with Python

```
In [2]: import pandas as pd
        from sqlalchemy import create_engine
```

```
In [8]: def retrieve_data():
        # 1. Credenciais de acesso
        usuario = 'looqbox-challenge'
        senha = 'looq-challenge'
        host = '35.199.115.174'
        porta = '3306' # porta padrão do MySQL
        nome_do_banco = 'looqbox-challenge'

        # 2. String de conexão (URL de conexão)
        # Formato: mysql+pymysql://usuario:senha@host:porta/nome_do_banco
        string_conexao = f'mysql+pymysql://{usuario}:{senha}@{host}:{porta}/{nome_do_banco}'

        # 3. Motor de conexão (engine)
        engine = create_engine(string_conexao)

        # 4. Consulta SQL
        query1 = """SELECT
            STORE_CODE,
            STORE_NAME,
            START_DATE,
            END_DATE,
            BUSINESS_NAME,
            BUSINESS_CODE
            FROM `looqbox-challenge`.`data_store_cad`;
        """

        query2 = """SELECT
            STORE_CODE,
            DATE,
```

```

        SALES_VALUE,
        SALES_QTY
    FROM `looqbox-challenge`.`data_store_sales`
    WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
    """

    df1 = pd.read_sql(query1, con=engine)
    df2 = pd.read_sql(query2, con=engine)

    # 6. retorna o DataFrame
    return df1, df2

```

```
In [9]: df1, df2 = retrieve_data()
```

```
In [10]: df1.head()
```

```
Out[10]:
```

	STORE_CODE	STORE_NAME	START_DATE	END_DATE	BUSINESS_NAME	BUSINESS_CODE
0	1	Sao Paulo	2006-10-01		Varejo	1
1	2	Chicago	2007-10-01		Varejo	1
2	3	Roma	2008-10-01		Varejo	1
3	4	Tokio	2009-10-01		Varejo	1
4	5	Paris	2019-01-01		Proximidade	2

```
In [11]: df2.head()
```

```
Out[11]:
```

	STORE_CODE	DATE	SALES_VALUE	SALES_QTY
0	1	2019-01-01	196623.22	12838
1	10	2019-01-01	126795.44	4933
2	11	2019-01-01	223937.00	7724
3	12	2019-01-01	200251.80	7043
4	13	2019-01-01	196623.22	12838

```
In [21]: df = pd.merge(df2, df1, on='STORE_CODE', how='left')
df = df[['STORE_NAME', 'BUSINESS_NAME', 'SALES_VALUE', 'SALES_QTY']]
df = df.groupby(['STORE_NAME', 'BUSINESS_NAME']).agg({'SALES_VALUE': 'sum', 'SALES_QTY': 'sum'}).reset_index()
df['TM'] = df['SALES_VALUE'] / df['SALES_QTY']
df = df.drop(columns=['SALES_VALUE', 'SALES_QTY'])
df.rename(columns={'STORE_NAME': 'Loja', 'BUSINESS_NAME': 'Categoria'}, inplace=True)
df['TM'] = df['TM'].round(2)
df

```

Out[21]:

	Loja	Categoria	TM
0	Bahia	Atacado	15.39
1	Bangkok	Posto	13.67
2	Belem	Proximidade	15.37
3	Berlin	Proximidade	15.39
4	Buenos Aires	Atacado	15.39
5	Chicago	Varejo	15.53
6	Dubai	Atacado	15.39
7	Hong Kong	Farma	26.33
8	London	Farma	28.96
9	Madri	Farma	29.00
10	Miami	Posto	13.67
11	New York	Proximidade	15.39
12	Paris	Proximidade	15.39
13	Rio de Janeiro	Farma	29.56
14	Roma	Varejo	15.39
15	Salvador	Atacado	15.39
16	Sao Paulo	Varejo	15.39
17	Sidney	Posto	13.67
18	Tokio	Varejo	15.39
19	Vancouver	Posto	13.67

Case 3

3) Building your own visualization

Create at least one chart using the table **IMDB_movies**. The code must be in Python, and you are free to use any libraries, data in the table and graphic format. Explain why you chose the visualization (or visualizations) you are submitting.

```
In [1]: import pandas as pd
from sqlalchemy import create_engine
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: def retrieve_data():
# 1. Credenciais de acesso
usuario = 'looqbox-challenge'
senha = 'looq-challenge'
host = '35.199.115.174'
porta = '3306' # porta padrão do MySQL
nome_do_banco = 'looqbox-challenge'

# 2. String de conexão (URL de conexão)
# Formato: mysql+pymysql://usuario:senha@host:porta/nome_do_banco
string_conexao = f'mysql+pymysql://{usuario}:{senha}@{host}:{porta}/{nome_do_banco}'

# 3. Motor de conexão (engine)
engine = create_engine(string_conexao)

# 4. Consulta SQL
query = "SELECT * FROM `looqbox-challenge`.`IMDB_movies`"
df = pd.read_sql(query, con=engine)

# 6. retorna o DataFrame
return df
```

```
In [3]: df = retrieve_data()
df.head()
```

Out[3]:

		Id	Title	Genre	Director	Actors	Year	Runtime	Rating	Votes	RevenueMillions	I
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.0	757074	333.0		
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.0		
2	3	Split	Horror,Thriller	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.0	157606	138.0		
3	4	Sing	Animation,Comedy,Family	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.0	60545	270.0		
4	5	Suicide Squad	Action,Adventure,Fantasy	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.0	393727	325.0		

```
In [4]: df.tail()
```


Out[4]:

		Id	Title	Genre	Director	Actors	Year	Runtime	Rating	Votes	RevenueMillions	Metascore
	995	996	Secret in Their Eyes	Crime,Drama,Mystery	Billy Ray	Chiwetel Ejiofor, Nicole Kidman, Julia Roberts...	2015	111	6.0	27585	NaN	41
	996	997	Hostel: Part II	Horror	Eli Roth	Lauren German, Heather Matarazzo, Bijou Philli...	2007	94	6.0	73152	18.0	41
	997	998	Step Up 2: The Streets	Drama,Music,Romance	Jon M. Chu	Robert Hoffman, Briana Evigan, Cassie Ventura,...	2008	98	6.0	70699	58.0	51
	998	999	Search Party	Adventure,Comedy	Scot Armstrong	Adam Pally, T.J. Miller, Thomas Middleditch,Sh...	2014	93	6.0	4881	NaN	21
	999	1000	Nine Lives	Comedy,Family,Fantasy	Barry Sonnenfeld	Kevin Spacey, Jennifer Garner, Robbie Amell,Ch...	2016	87	5.0	12435	20.0	11

In [5]:

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
Column Non-Null Count Dtype
--- -
0 Id 1000 non-null int64
1 Title 1000 non-null object
2 Genre 1000 non-null object
3 Director 1000 non-null object
4 Actors 1000 non-null object
5 Year 1000 non-null int64
6 Runtime 1000 non-null int64
7 Rating 1000 non-null float64
8 Votes 1000 non-null int64
9 RevenueMillions 872 non-null float64
10 Metascore 936 non-null float64
dtypes: float64(3), int64(4), object(4)
memory usage: 86.1+ KB

In [6]:

#Média de Avaliação dos filmes por ano de lançamento
df = df.dropna(subset=['Rating', 'Year']) *# Remove linhas com valores nulos em 'Rating' ou 'Year'*
df_years = df.groupby('Year').agg({'Rating': 'mean', 'Id': 'count'}).reset_index()
df_years

Out[6]:

	Year	Rating	Id
0	2006	7.272727	44
1	2007	7.207547	53
2	2008	6.846154	52
3	2009	7.058824	51
4	2010	6.883333	60
5	2011	6.888889	63
6	2012	6.937500	64
7	2013	6.857143	91
8	2014	6.887755	98
9	2015	6.629921	127
10	2016	6.474747	297

In [7]:

tema do seaborn
sns.set_theme(style="white")
fig, ax1 = plt.subplots(figsize=(12, 6))

```
# Certifique-se de que os dados estão ordenados por ano
df_years = df_years.sort_values('Year')

# --- GRÁFICO 1: Quantidade de Filmes (Barras) ---
ax1.bar(df_years['Year'], df_years['Id'], color='royalblue', alpha=0.6, width=0.6, label='Quantidade de Filmes')
ax1.set_xlabel('Year', fontsize=12)
ax1.set_ylabel('Quantidade de Filmes', color='b', fontsize=12)
ax1.tick_params(axis='y', labelcolor='b')

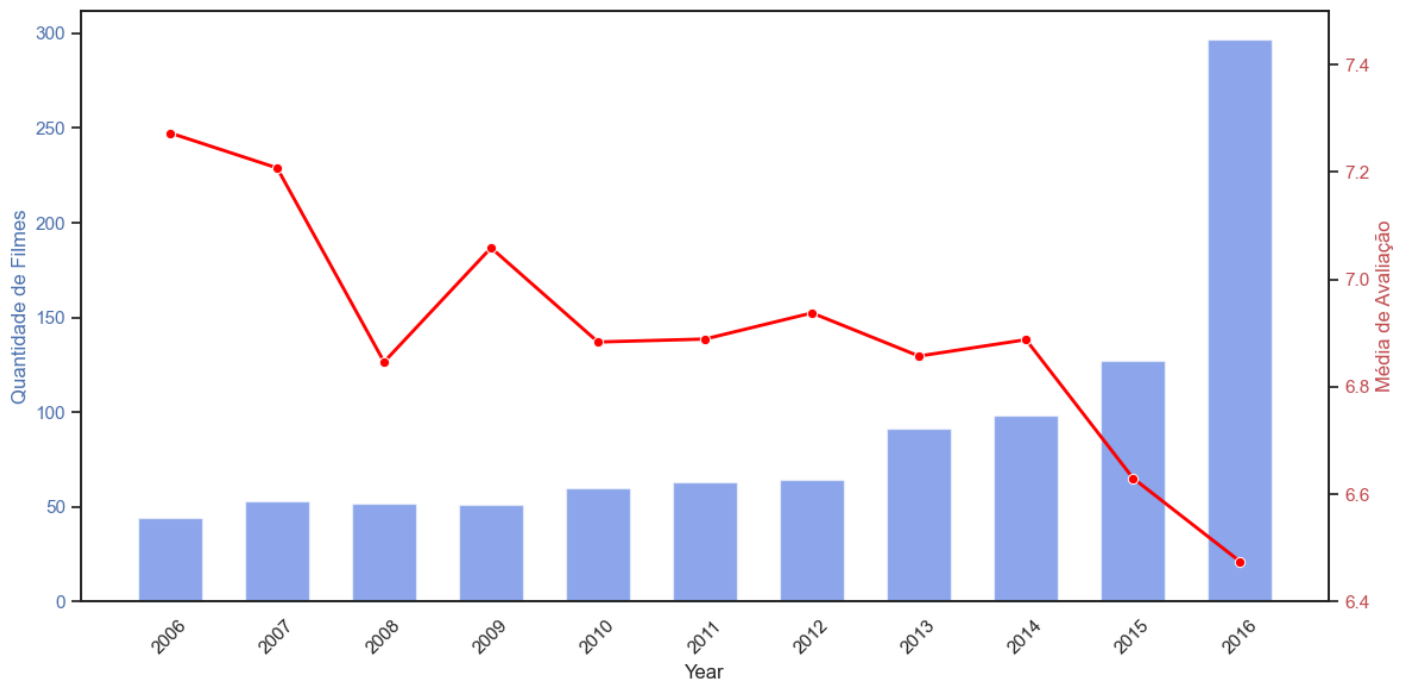
plt.xticks(df_years['Year'], rotation=45)

# --- GRÁFICO 2: Média de Avaliação (Linha no Eixo secundário) ---
ax2 = ax1.twinx() # Cria o segundo eixo Y compartilhando o mesmo eixo X numérico
sns.lineplot(data=df_years, x='Year', y='Rating', ax=ax2, color='red', marker='o', linewidth=2)
ax2.set_ylabel('Média de Avaliação', color='r', fontsize=12)
ax2.tick_params(axis='y', labelcolor='r')

# Remove as linhas de grade de ambos para não sobrepor o visual
ax1.grid(False)
ax2.grid(False)

#ajusta o eixo da avaliação de 0 a 10
ax2.set_ylim(6.4, 7.5)

plt.tight_layout()
plt.show()
```



Nesse gráfico podemos verificar a evolução da quantidade de filmes lançados anualmente, porém temos um piora na qualidade dos filmes que pode ser verificada pela linha vermelha no gráfico. A queda nas avaliações entre 2006 e 2016 foi de 7.3 para 6.5, o que representa uma queda percentual de 12%.

```
In [8]: (df_years.Rating.max()-df_years.Rating.min())/df_years.Rating.min()
```

```
Out[8]: 0.12324492979719182
```

```
In [ ]:
```